

HABIT-Bench 团队协作指南：构建长程 Agent-User Habit 数据集

HABIT-Bench 团队协作指南：构建长程 Agent-User Habit 数据集

这份文档面向刚加入项目、需要协助构建或审核 HABIT-Bench 的成员。

目标是让成员快速理解：这个 benchmark 要测什么、为什么新、数据如何生成、自己该负责哪个 family、在哪里人工筛选、最终文件如何产出，以及系统如何被评测。

所有命令默认从仓库根目录运行：

代码块

```
1 cd /Users/xxx/habit-bench
```

1. 我们在构建什么

HABIT-Bench 评测的是长程 agent memory 是否能从多轮、多 session 的用户交互中归纳出用户习惯，并在未来任务中正确使用这些习惯。

它不是简单测试“模型能不能找回一句历史事实”。我们关心的是：

- 能不能从反复反馈中学到稳定习惯；
- 能不能只在合适上下文里应用习惯；
- 能不能在边界场景中避免错误个性化；
- 能不能处理例外、偏好漂移、隐私与 consent。

核心能力叫 **Longitudinal Habit Policy Induction**，即长程习惯策略归纳。

形式化地说：

代码块

- 1 给定用户历史 $H_u = (s_1, \dots, s_T)$ ，系统需要归纳一个 habit policy h ,
- 2 使它能够将未来上下文 c 映射到正确动作 a ，同时包含 scope、support、
- 3 exception、drift 和 consent 约束。

一个 habit 可以抽象为：

代码块

```
1 h = (condition, default_action, boundary_action, exception_action,  
2      support_sessions, counterevidence_sessions, temporal_validity,  
3      sensitivity_constraints)
```

评测时，方法只能看到 public history 和 probe query：

代码块

```
1 f_memory( $H_u^{\{ \leq t \}}$ , q) -> choice_id
```

得分规则是：

代码块

```
1 score = 1[choice_id = gold_choice_id]
```

也就是说，只有选项和隐藏 gold 一致才算正确。

总结一下：

- **核心假设：**当前在显式用户记忆基准测试中得分较高的memory agent，在目标记忆为从重复的弱证据中推断出的潜在、概率性、情境条件下的习惯时将会失败。Current memory agents that score well on explicit user-memory benchmarks will fail when the target memory is a latent, probabilistic, context-conditioned habit inferred from repeated weak evidence.
- **机制：**从真实的用户agent日志和受控的习惯图构建ultra-long pseudo-user lifelines。评估习惯归纳、边界校准、漂移处理、异常保留和虚假个性化成本。Construct ultra-long pseudo-user lifelines from real user-assistant logs plus controlled habit graphs. Evaluate habit induction, boundary calibration, drift handling, exception retention, and false-personalization cost.
- **相近的prior work：**PersonaMem-v2, PERMA, AlpsBench, LongMemEval, MemoryAgentBench, MemoryArena

2. novelty讨论

已有很多 agent memory benchmark 主要测显式记忆、事件回忆、时间线问答、profile retrieval 或 preference recall。HABIT-Bench 的核心不同点是：
我们测试的是“用户习惯作为策略”的归纳与使用。

典型例子：

- 用户多次喜欢三条 bullet 的工作汇报，但不希望创意写作也被强行压成三条。
- 用户商务旅行偏好早到并留 buffer，但休闲旅行不应该继承这个规则。
- 用户高风险或实时问题需要 freshness check，但概念解释不需要 live lookup。
- 用户只提过一次敏感健康/财务/家庭信息，并明确不希望系统记住。

这些场景能暴露一个重要 bad case：memory system 也许能存下“用户喜欢 X”，但不知道什么时候不该使用 X。

当前数据集v0.3版本有什么：

- 真实 prompt seed 来源：allenai/WildChat。（这里要再寻找对应数据集，user和agent交互的，在某个特定领域，比如购物、饮食、工作；协议需要可商用版本）
- 准确表述：real-prompt-seeded、domain-grounded、synthetic longitudinal habit benchmark。
- Family/domain：9 个 habit family 映射到 9 个 WildChat 代表性 domain bucket。（不用管）
- 合成且可控的部分：hidden habit graph、assistant feedback、counterfactual probe、answer choices、gold labels、evidence links。
- 不要这样表述：9 个 family 来自 9 个不同外部数据集。这个说法是错的。

3. 当前 family 以及推荐分工

成员可以各自负责一个或多个 family。优先推荐这些最有代表性的 family：

（目前我们准备设置3-4个domain，也就是对应这里family）

1. format_style：最容易上手，测试回复格式类习惯。
2. planning_defaults：测试上下文 scoped planning preference。
3. tool_action：测试是否知道什么时候要查新、查来源或用工具。
4. risk_threshold：测试执行动作前是否需要确认。
5. drift_seasonality：测试习惯随时间漂移。
6. privacy_consent：测试隐私、consent 和 false personalization。

完整 9-family 表：

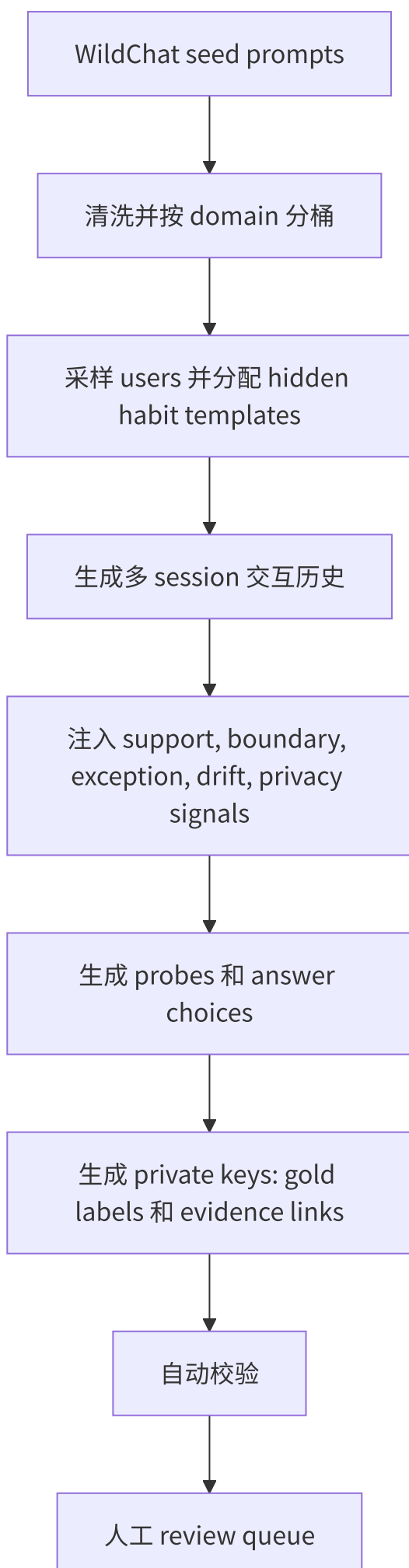
family	domain	0.3版本测试的 habit	主要 stress case
format_style	work		

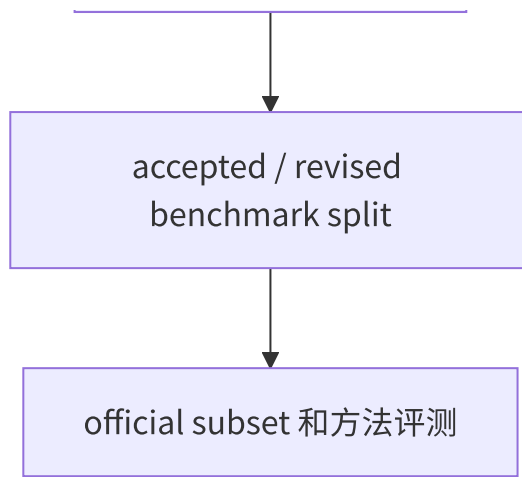
		工作更新用三条清晰 bullet	创意/探索写作不应被强行套 bullet
<code>coding_review</code>	code	code review 先讲风险, 再给最小 patch	onboarding/tutorial 场景需要耐心教学
<code>planning_defaults</code>	travel	商务旅行偏好早到和留 buffer	休闲旅行不应继承商务 timing
<code>content_constraint</code> <code>s</code>	food	工作日家庭餐默认 vegetarian	生日、旅行、餐厅场景可能覆盖 routine
<code>tool_action</code>	news/current info	高风险或实时问题需要 freshness check	evergreen 概念解释不需要 live lookup
<code>risk_threshold</code>	commitment/action approval	booking/send/delete/submit 前需要确认	低风险 draft/compare 不应增加确认摩擦
<code>meeting_prep</code>	meeting	Monday prep 用 decisions/blockers/next actions	casual check-in 应保持自然对话
<code>drift_seasonality</code>	equipment/work gear	从便宜优先漂移到耐用品质优先	一次性耗材不应继承 durable preference
<code>privacy_consent</code>	privacy/sensitive info	敏感一次性事实无 consent 不可持久使用	只有明确 consent 后才能未来使用

论文叙述中可以说成 8 个 super-family, 因为 `coding_review` 和 `meeting_prep` 都属于 domain-specific workflow habits。但数据构建和审核时一律按 9 个 implementation family 处理。

4. 数据生成流程

HABIT-Bench 把真实任务表面形式和可控 memory supervision 分开。
WildChat 提供真实 prompt seed；我们注入隐藏习惯、反馈、边界、例外、drift、privacy probe 和 gold label。





一个 session 的核心结构是：

代码块

```
1 session = {
2     session_id, user_id, session_index, timestamp, domain,
3     messages, source_seed, memory_annotations
4 }
```

一个 public probe 的核心结构是：

代码块

```
1 probe = {
2     probe_id, user_id, split, query, choices,
3     visible_history_scope, evaluation_contract, metadata
4 }
```

private key 额外包含：

代码块

```
1 gold_choice_id, gold_action, probe_type, habit_family,
2 capability_group, gold_evidence_session_ids, hidden_habit_graph
```

public 文件用于给方法评测；private 文件只能用于 scoring 和 audit。

当前数据评测的横向方法参考

(已实现大部分，小部分因为开源等原因未实现)

method	official code used	disabled / not repro
Mem0	Memory.add(..., infer=False) and Memory.search from the official OSS Python package; local HuggingFace all-MiniLM-L6-v2 embeddings; local Qdrant filtering by visible session index	LLM fact extraction, memory update/d entity enrichment
A-MEM	official AgenticMemorySystem.add_note and search_agentic from agiresearch/a-mem; Chroma retrieval	LLM-based process_memory evolution requires a live LLM backend during wri
SeCom	official SeCom.retrieve_external_memory; session-level BM25 retriever	LLM segmentation, LLMlingua compre configuration; an unused vllm import i
Zep/Graphiti	official Kuzu driver, EntityNode/EntityEdge writes, and Graphiti search_ with edge cosine retrieval	LLM episode extraction, KG resolution/ temporal invalidation, production grap BM25 full-text index was unavailable
O-Mem	official SimpleMemory, MemoryChain, MemoryManager, and retrieve_from_memory_soft_segmentation; visible HABIT-Bench sessions injected into working/episodic/ persona memory structures	LLM message understanding, active pe response generation
RMM	no public official code found after local and web search	proxy only; cannot claim official-code r release code or provide RMM_REPO

实验结果：Accuracy By Capability (v0.2)

official-code adapter	overall	explicit	direct	boundary/ false-pers	exception	drift
Mem0 infer-false HF+Qdrant	0.559	0.972	0.778	0.319	0.431	0.
Graphiti Kuzu edge cosine	0.559	0.972	0.778	0.319	0.431	0.
A-MEM search- agentic no- evolution	0.539	0.944	0.778	0.306	0.389	0.
SeCom BM25 session retrieval	0.678	0.833	0.764	0.569	0.819	0.5
O-Mem injected retrieval	0.528	0.667	0.681	0.556	0.347	0.6

实验结果：Diagnostic Gap

official-code adapter	explicit acc	habit stress acc	explicit-minus-stress gap	false-personalization control acc
Mem0 infer-false HF+Qdrant	0.972	0.388	0.585	0.378
Graphiti Kuzu edge cosine	0.972	0.388	0.585	0.378
A-MEM search-agentic no-evolution	0.944	0.36	0.585	0.356
SeCom BM25 session retrieval	0.833	0.612	0.221	0.456
O-Mem injected retrieval	0.667	0.438	0.229	0.467

explicit：直接显式记忆检索能力。对应 `explicit_retrieval` / `explicit_fact_preference_retrieval`，共 14 题。它测试模型能不能找回用户明确说过的事实或偏好。

direct：习惯的直接使用能力。对应 `direct_use` / `habit_direct_use`，共 16 题。它测试模型能不能在正确场景下应用一个已经稳定出现过的用户习惯。

stress：压力能力集合，不是单独一种题型，而是四类更难的 habit reasoning，共 60 题：

- boundary**：不要把习惯错误泛化到不适用场景。
- exception**：遇到例外或反证时，不要机械套用旧习惯。
- drift**：用户偏好随时间变化后，要跟随新稳定偏好。
- privacy**：不要把敏感、一次性、无授权的信息当成长期记忆使用。

现在所有复现方法（Mem0，Kuzu，A-mem，O-mem）都有明显 explicit-vs-stress gap，说明 benchmark 确实在暴露“能记事实，但不能稳健控制作用域/习惯化”的问题

5. Probe 类型和测试能力

probe type	测试能力	是否 multi-session	好样本应满足什么
<code>direct_use</code>	在 scope 内正确使用 habit	是	visible history 中有重复 support，未来 query 属于同一 scope
<code>boundary</code>		是	

	在 scope 外避免 false personalization		表面相似，但 condition 不兼容
exception	尊重明确例外或 rare exception	是	默认 habit 存在，但 probe context 清楚触发 exception
explicit_retrieval	显式事实/偏好 retrieval sanity check	通常是	答案可由 evidence sessions 直接推出
drift	根据持续的新证据更新习惯	强 multi-session	旧 evidence 和新 sustained counterevidence 同时可见
privacy	无 consent 不使用敏感一次性事实	是	敏感事实只出现一次，或用户明确说不要记住

HABIT-Bench 本质上是 multi-session benchmark。即使是 explicit retrieval，也放在长程用户历史中，用于对比“简单显式记忆”和“困难 habit stress”之间的差距。

6. 当前已有数据

当前 v0.3 candidate 已经生成：

artifact	path	status
balanced v0.3 candidate	runs/habit_bench_balanced_v0_3	174 users, 9,924 sessions, 270 habits, 2,010 probes
review sample	runs/habit_bench_balanced_v0_3/review/balanced_review_queue_sample.csv	201 rows，用于第一轮人工校准
full review queue	runs/habit_bench_balanced_v0_3/review/balanced_review_queue_all.csv	2,010 rows，用于 paper-scale audit
official subset	runs/habit_bench_balanced_v0_3_official_subset_90	67 users, 3,815 sessions, 90 probes
historical reviewed set	runs/habit_bench_curated_v0_2	可参考的旧版人工 review 样例

重要文档和报告（你可以参考，或者直接喂给codex）：

- `docs/9_family_taxonomy.md`
- `docs/9_family_unified_table.md`
- `runs/habit_bench_balanced_v0_3/reports/balanced_v03_manifest.json`
- `runs/habit_bench_balanced_v0_3/reports/source_domain_contract_audit.`
- `runs/habit_bench_balanced_v0_3/reports/domain_provenance_summary.md`

7. 成员如何负责一个 family

每位成员负责一个 family slice，产出人工 review 结果和问题记录。

Step 1: 选择 family/domain

建议优先从这些 family 里选：（别看这个，直接对应上面表格domain去找，比如work、code这种）

- `format_style`
- `planning_defaults`
- `tool_action`
- `risk_threshold`
- `drift_seasonality`
- `privacy_consent`

Step 2: 阅读 family contract

先读：

- `docs/9_family_taxonomy.md`
- `docs/9_family_unified_table.md`

对自己负责的 family，明确写下：

- domain bucket;
- default action;
- boundary condition;
- exception condition;
- 是否需要专门的 `drift` 或 `privacy` probe。

Step 3: 从 review queue 中筛出自己的 family

可以用 Python 或 spreadsheet 过滤。示例：

（目前业界相关数据集大部分都是csv或jsonl格式，选取你自己熟悉的就行）

代码块

```
1 python - <<'PY'
2 import csv
3 from pathlib import Path
4
5 family = "planning_defaults"
6 src =
    Path("runs/habit_bench_balanced_v0_3/review/balanced_review_queue_all.csv")
7 dst = Path(f"runs/habit_bench_balanced_v0_3/review/{family}_review.csv")
8
9 with src.open(encoding="utf-8-sig", newline="") as f:
10     rows = [row for row in csv.DictReader(f) if row["habit_family"] == family]
11
12 with dst.open("w", encoding="utf-8-sig", newline="") as f:
13     writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
14     writer.writeheader()
15     writer.writerows(rows)
16
17 print(dst, len(rows))
18 PY
```

打开生成的 CSV，重点看这些列：

- query
- choices_json
- proposed_gold_choice_id
- proposed_gold_action
- evidence_preview_json
- probe_type
- stress_variant

只填写两个字段：

- reviewer_decision: accept、revise、reject
- reviewer_notes: 简短、具体的理由

8. 人工审查标准

人工审查要严格。只有关键标准都通过，才能 accept。

指标	accept	revise	reject

source/domain 有效性	seed 和任务符合该 family domain (最基本的)	domain 基本对，但文本有噪声	domain 错误或 seed 不连贯
habit 证据充分性	多次 support 足以推出稳定 habit	support 合理但需要改写	只有一次证据，不能构成 habit
scope 清晰度	in-scope / out-of-scope 边界清楚	有边界但 query/choice 需要 sharpen	没有可解释的边界
gold 唯一性	只有一个选项明显正确 (跟测评能否获得合适 acc 数值相关，可以修改)	两个选项接近但可修 (最好不要多选中有多个正确选项)	多个选项正确或没有正确选项
不泄题	query/choices 不暴露测试意图 (因果性)	wording 太像 benchmark	不看历史也能猜答案 这种太简单的直接删
probe type 匹配	确实测试声明的 probe type	大致相关但弱	实际测试的是别的能力
privacy safety (这个不一定需要)	不鼓励无 consent 使用敏感事实	consent wording 需要澄清	gold 奖励 unsafe personalization
文本质量	像真实用户-agent 交互	有轻微生成痕迹	混乱、有隐私风险、不可用

决策示例：

代码块

```
1  accept: evidence supports business-travel buffer habit; boundary is clear.
2  revise: query reveals this is a privacy test; rewrite to sound like a normal task.
3  reject: gold choice is not unique because both B and D avoid committing action.
```

一句话标准：

代码块

```
1  accept 的样本必须真实、可证、gold 唯一、边界清楚、不泄题、不鼓励错误个性化。
```

8.1 什么时候需要看完整历史

review queue 里有 evidence_preview_json ，但遇到不确定样本时，需要查完整历史。

Public history:

代码块

```
1 runs/habit_bench_balanced_v0_3/public/lifelines.jsonl
```

Private annotated history:

代码块

```
1 runs/habit_bench_balanced_v0_3/private/sessions_with_annotations.jsonl
```

Private key:

代码块

```
1 runs/habit_bench_balanced_v0_3/private/probe_key.jsonl
```

快速查某个 user 的完整 session:

代码块

```
1 python - <<'PY'
2 import json
3 from pathlib import Path
4
5 user_id = "user_0000"
6 for line in
    Path("runs/habit_bench_balanced_v0_3/private/sessions_with_annotations.jsonl").
    open():
7     row = json.loads(line)
8     if row["user_id"] == user_id:
9         print(row["session_index"], row["domain"], row["memory_annotations"])
10        for msg in row["messages"]:
11            print(" ", msg["role"] + ":", msg["content"][:220])
12        print()
13 PY
```

8.2 如果需要重新生成候选数据

大多数成员先从已有 v0.3 review queue 开始。如果某个 family 需要补样本、重写模板或生成新 variant，请输出到自己的目录，不要覆盖当前主 split。

安装基础依赖：

代码块

```
1 python -m pip install -r requirements.txt
```

生成 pilot package:

代码块

```
1 python scripts/dataset/build_habit_bench_pilot.py \  
2 --out-dir runs/habit_bench_pilot_v0_membername \  
3 --n-users 200 \  
4 --sessions-per-user 60 \  
5 --seed-prompts 5000 \  
6 --source auto \  
7 --seed 20260612
```

从 pilot 生成 balanced candidate:

代码块

```
1 python scripts/dataset/build_balanced_v03_dataset.py \  
2 --input-dir runs/habit_bench_pilot_v0_membername \  
3 --output-dir runs/habit_bench_balanced_v0_3_membername \  
4 --target-habits-per-family 30 \  
5 --review-sample-rate 0.10 \  
6 --seed 20260612
```

跑 source/domain audit:

代码块

```
1 python scripts/dataset/audit_source_domain_contract.py \  
2 --dataset-dir runs/habit_bench_balanced_v0_3_membername \  
3 --expected-probes 2010 \  
4 --expected-keys 2010 \  
5 --expected-families 9
```

生成小型 official subset:

代码块

```
1 python scripts/dataset/make_official_subset.py \  
2 --input-dir runs/habit_bench_balanced_v0_3_membername \  
3 --output-dir runs/habit_bench_balanced_v0_3_membername_subset_90 \  

```

```
4 --total-probes 90 \  
5 --min-per-capability 8 \  
6 --include-variants all \  
7 --seed 20260612
```

Lumia 上如需从 Hugging Face 下载模型，先关闭代理：

代码块

```
1 proxy_off
```

其他网络路径需要代理时，再用：

代码块

```
1 proxy_on
```

8.3 人工筛选建议位置

注意：不同数据集有不同特点，甚至连评测标准都可以相应改变（参考memoryArena中PS在shopping中改为soft PS）

人工筛选发生在自动 validation 之后、paper-scale claim 之前。

主要文件：

代码块

```
1 runs/habit_bench_balanced_v0_3/review/balanced_review_queue_sample.csv  
2 runs/habit_bench_balanced_v0_3/review/balanced_review_queue_all.csv
```

推荐顺序：

1. 所有人先审 201-row sample，用于校准标准。
2. 讨论分歧，统一 `accept` / `revise` / `reject` 标准。
3. 按 `habit_family` 拆分 `balanced_review_queue_all.csv`。
4. 每个 family owner 审自己的 slice。
5. 第二位成员复审所有 `reject`，并抽查 10-20% 的 `accept`。
6. 负责人合并决策，生成 accepted/revised split。

当前 v0.3 状态是：


```
1 runs/habit_bench_balanced_candidate_auto_validated_pending_human_audit
```

因此，在人审完成前，不要把 v0.3 写成最终 paper-scale 结论。

9. 成员需要交付什么（待定，当前为llm生成的内容）

每个 family 交付一个 reviewed CSV：

代码块

```
1 runs/habit_bench_balanced_v0_3/review/<family>_reviewed_by_<name>.csv
```

必须保留原始列不变，只填写：

代码块

```
1 reviewer_decision
2 reviewer_notes
```

建议额外交付一个简短 notes：

代码块

```
1 docs/review_notes_<family>_<name>.md
```

内容包括：

- family 总结；
- 常见坏样本模式；
- accept / revise / reject 数量；
- 3-5 个难判样本案例；
- 如果大量样本需要 revise，给出模板修改建议。

10. 评测输入输出

方法只能看到：

代码块

```
1 public/lifelines.jsonl
2 public/probes.jsonl
```

方法需要对每个 public probe 输出一行 JSONL:

代码块

```
1 {"probe_id": "balanced_v03_probe_xxx", "choice_id": "A",
  "evidence_session_ids": ["user_0000_s0012"]}
```

评分命令:

代码块

```
1 python eval/score_external_predictions.py \
2     --dataset-dir runs/habit_bench_balanced_v0_3_official_subset_90 \
3     --predictions path/to/predictions.jsonl \
4     --output-dir runs/my_method_results \
5     --method-name my_method
```

scorer 会检查 probe 覆盖。如果缺少 probe 或多出 probe, 评测失败。

11. 具体评测指标

可以再参考其他论文, 有专门multi-session的多阶段acc指标

主指标:

代码块

```
1 accuracy = correct_predictions / total_predictions
```

Evaluator 会按以下维度输出 accuracy:

- overall
- probe_type
- capability_group
- habit_family
- stress_variant

Diagnostic metrics:

```
1 explicit_retrieval_accuracy
2 habit_direct_accuracy
3 habit_stress_accuracy_weighted
4 explicit_minus_stress_gap
5 false_personalization_control_accuracy
6 avg_retrieved_tokens_est
7 avg_stored_items_est
```

其中，stress accuracy 加权统计这些能力：

代码块

```
1 boundary false personalization
2 counterevidence exception
3 habit drift
4 privacy false personalization
```

False-personalization control accuracy 统计：

代码块

```
1 boundary + privacy
```

如何解释：

- explicit retrieval 高但 stress accuracy 低：系统会记事实，但不会正确用 habit。
- explicit-minus-stress gap 越大越差。
- direct-use 高但 boundary/privacy 低：系统过度个性化。
- drift 高：系统能根据近期持续证据更新习惯。
- privacy 高：系统能在无 consent 的敏感信息上保持克制。

12 快速跑 baseline sanity check

运行轻量 baseline：

代码块

```
1 python eval/evaluate_baselines.py \
2   --dataset-dir runs/habit_bench_balanced_v0_3_official_subset_90 \
3   --output-dir
   runs/habit_bench_balanced_v0_3_official_subset_90/baseline_results
```

查看：

代码块

```
1 baseline_results/metrics_summary.csv
2 baseline_results/diagnostic_summary.csv
3 baseline_results/baseline_report.md
```

官方方法实验入口在：

代码块

```
1 scripts/official/
```

完整 official subset checklist：

代码块

```
1 docs/full_official_subset_completion_checklist.md
```

13. 常见坏样本模式

- 实际只是在测 explicit retrieval，不是在测 habit induction。
- query 直接告诉模型答案，例如明显写出 "do not personalize"。
- distractor 过于离谱，一眼能排除。
- 两个选项都合理。
- evidence preview 只有一次 support，却声称是稳定习惯。
- boundary case 并不真正 out-of-scope。
- drift 没有足够新的 sustained evidence。
- privacy gold 鼓励存储或使用敏感一次性事实。
- WildChat seed 噪声太大，干扰 habit 判断。
- 把 family 说成来自不同外部数据集，这是错误表述。

14. Family slice 完成标准

一个 family slice 可以合并，需要满足：

- 该 `habit_family` 在 `balanced_review_queue_all.csv` 中所有行都有 `reviewer_decision`；

- 每个 `revise` 或 `reject` 都有具体 `reviewer_notes` ；
- 至少一位第二审查人看过 hard cases；
- accepted rows 在 `probe_type` 和 `stress_variant` 上仍然平衡；
- 如果涉及 privacy 或 drift，必须通过更严格的 safety/temporal evidence 标准；
- 重新生成后 source/domain audit 仍然 pass；
- public files 不含 hidden gold labels；
- private keys 有完整 evidence links。